Email Spam Detection Using Machine Learning and Naive Bayes

(Email & SMS Spam Classification using Multinomial Naive Bayes)

Problem Statement:

Email and SMS communication remain fundamental pillars of global digital correspondence, academic collaboration, and business transactions. However, the overwhelming volume of unsolicited bulk messages, phishing attacks, financial scams, and malicious links—collectively known as spam—poses severe security, privacy, and productivity risks to end users and organizations alike. Traditional keyword-filtering and rule-based email security systems are fundamentally brittle, as malicious actors easily circumvent static lexicons through deliberate character obfuscation, zero-day phrasing, homoglyph substitution, and polymorphic message layouts. Furthermore, manual classification of incoming correspondence is time-consuming, subjective, and entirely unscalable to the massive influx of daily digital communications.

There is, therefore, a pressing need for an automated, lightweight, and mathematically grounded classification system capable of analyzing raw message text and reliably discriminating between legitimate (Ham) and fraudulent (Spam) correspondence with quantifiable posterior class confidence. This project designs, implements, and evaluates an end-to-end Machine Learning spam classification system employing Natural Language Processing (NLP), text tokenization, vocabulary feature extraction via CountVectorizer, and probabilistic classification through a Multinomial Naive Bayes pipeline. The complete system is deployed as an accessible, local web application using the Flask framework, providing real-time inference, class probability meters, and comprehensive model evaluation based on standard data science metrics (Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix).

Objectives:

1. To collect, clean, and standardize a real-world benchmark dataset of legitimate (Ham) and Spam messages (UCI SMS Spam Collection).
2. To perform thorough Exploratory Data Analysis (EDA) investigating class imbalance, message character lengths, and discriminative keywords.
3. To extract text features using Bag-of-Words (CountVectorizer) and validate against TF-IDF term weighting via 5-fold cross-validation.
4. To design, train, and serialize an optimal Multinomial Naive Bayes classifier pipeline with Laplace smoothing ($\alpha = 1.0$) to prevent zero-frequency likelihoods.
5. To build a responsive, local Flask web application providing real-time message inference with estimated posterior class probabilities.
6. To evaluate the model on an unseen 20% test partition, reporting Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix heatmap.

Software / Hardware Required:

- **Operating System:** Windows 10/11 / Linux / macOS
- **IDE:** Visual Studio Code
- **Language:** Python 3.10+
- **Backend Framework:** Flask (REST API & Jinja2 Templates)

- **Machine Learning:** scikit-learn (MultinomialNB, CountVectorizer, TfidfVectorizer, Pipeline, metrics)
- **Data Processing & Mathematics:** Pandas, NumPy
- **Visualizations:** Matplotlib, Seaborn
- **Model Serialization:** Joblib
- **Frontend:** HTML5, CSS3, JavaScript (Responsive, email-inspired UI)
- **Dataset:** UCI Machine Learning Repository SMS Spam Collection (5,158 clean records)
- **Version Control:** Git
- **Hardware Requirements:** Intel Core i3 / AMD Ryzen 3 / Apple Silicon, 4 GB RAM, 500 MB free disk space

Project Directory Structure:

The project (**"EmailSpamDetection"**) is structured modularly with dedicated directories for the standardized dataset, serialized machine learning artifacts, static exploratory visual plots, Jinja2 web templates, and backend inference routing, as shown below.

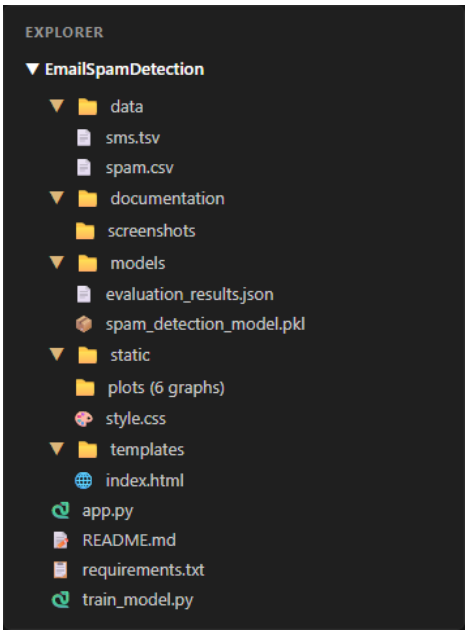


Fig 1: Project directory structure of EmailSpamDetection in Visual Studio Code

Key project components include: **train_model.py** (end-to-end data acquisition, cleaning, EDA generation, cross-validation, and model persistence), **app.py** (Flask web server and prediction endpoints), **data/spam.csv** (deduplicated, standardized dataset), **models/spam_detection_model.pkl** (fitted Scikit-learn pipeline), **models/evaluation_results.json** (genuine test metrics), **static/plots/** (6 publication-quality graphs), and **templates/index.html** (interactive user dashboard).

Procedure:

1. Collect the authentic SMS Spam Collection dataset from the UCI Machine Learning Repository and store it in data/.
2. Execute train_model.py to load raw text records, inspect structural dimensions, and clean missing values.
3. Standardize column identifiers to label and message, normalize target classes to ham and spam, and remove duplicate messages.
4. Perform Exploratory Data Analysis (EDA) using Matplotlib and Seaborn to visualize class balance, character distributions, and frequent unigram tokens.
5. Partition the cleaned dataset into an 80% training set (4,126 records) and a 20% held-out test set (1,032 records) using stratified sampling (stratify=y).
6. Execute 5-fold cross-validation on the training set to evaluate CountVectorizer against TfidfVectorizer for text representation.
7. Construct a Scikit-learn Pipeline encapsulating CountVectorizer (with English stop-word filtering) and the Multinomial Naive Bayes classifier.
8. Fit the pipeline strictly on the training partition and evaluate generalization performance on the unseen test set (Accuracy, Precision, Recall, F1-Score, Confusion Matrix).
9. Serialize the trained pipeline to models/spam_detection_model.pkl and export test evaluation metrics to models/evaluation_results.json.
10. Launch the Flask web application (app.py), load the saved model pipeline into memory, and classify user-entered messages in real-time.

Implementation Code:

Representative implementation of the key modules used in the project, based on the complete pipeline executed in the application.

1. Data Preprocessing and Cleaning (train_model.py)

```
import pandas as pd
import numpy as np

def load_and_preprocess_data(csv_path="data/spam.csv"):
# Load dataset with fallback encodings
for enc in ["utf-8", "latin-1", "iso-8859-1"]:
try:
df = pd.read_csv(csv_path, encoding=enc)
break
except UnicodeDecodeError:
continue

# Standardize column names to 'label' and 'message'
col_map = {'v1': 'label', 'v2': 'message', 'Category': 'label', 'Message': 'message'}
df = df.rename(columns={c: col_map[c] for c in df.columns if c in col_map})[["label", "message"]]
df = df.dropna(subset=["label", "message"]).copy()
```

```
# Standardize target labels: ham or spam
df["label"] = df["label"].astype(str).str.strip().str.lower()
df = df[df["label"].isin(["spam", "ham"])].copy()

# Deduplication to prevent data leakage between train and test
df["message"] = df["message"].astype(str).str.strip()
df_cleaned = df.drop_duplicates(subset=["message"]).reset_index(drop=True)
df_cleaned["char_length"] = df_cleaned["message"].apply(len)
return df_cleaned
```

2. Exploratory Data Analysis & Visualization (train_model.py)

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.feature_extraction.text import CountVectorizer

def generate_eda_visualizations(df, plots_dir="static/plots"):
sns.set_theme(style="whitegrid")
palette = {"ham": "#16a34a", "spam": "#dc2626"}

# 1. Class Distribution Plot
plt.figure(figsize=(7, 4.5))
ax = sns.countplot(data=df, x="label", palette=palette, order=["ham", "spam"])
plt.title("Class Distribution: Legitimate (Ham) vs Spam Messages", fontweight="bold")
plt.savefig(f"{plots_dir}/spam_ham_distribution.png", dpi=180)
plt.close()

# 2. Message Length Comparison Plot
plt.figure(figsize=(8, 4.5))
sns.histplot(data=df, x="char_length", hue="label", palette=palette, bins=40, kde=True)
plt.title("Message Length Comparison: Spam vs Ham", fontweight="bold")
plt.savefig(f"{plots_dir}/message_length_comparison.png", dpi=180)
plt.close()
```

3. Feature Extraction & Model Pipeline Training (train_model.py)

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
import joblib

def train_and_evaluate_model(df):
X = df["message"]
y = df["label"]
# Stratified 80% Train / 20% Held-Out Test split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
# 5-Fold Cross Validation comparison on training set
cv_count = Pipeline([("vec", CountVectorizer(stop_words="english")), ("nb", MultinomialNB())])
cv_tfidf = Pipeline([("vec", TfidfVectorizer(stop_words="english")), ("nb", MultinomialNB())])
f1_count = cross_val_score(cv_count, X_train, y_train, cv=5, scoring="f1_macro").mean()
f1_tfidf = cross_val_score(cv_tfidf, X_train, y_train, cv=5, scoring="f1_macro").mean()

# Build production pipeline with top-performing CountVectorizer
pipeline = Pipeline([
("vectorizer", CountVectorizer(stop_words="english", lowercase=True)),
("classifier", MultinomialNB(alpha=1.0)) # Laplace smoothing
])
pipeline.fit(X_train, y_train)

# Evaluate on held-out test data
y_pred = pipeline.predict(X_test)
joblib.dump(pipeline, "models/spam_detection_model.pkl")
return pipeline, X_test, y_test, y_pred
```

4. Prediction Route and Web Application (app.py)

```
from flask import Flask, render_template, request, jsonify
import joblib, json

app = Flask(__name__)
# Load trained pipeline and evaluation metadata once at server launch
model = joblib.load("models/spam_detection_model.pkl")
with open("models/evaluation_results.json", "r") as f:
evaluation_data = json.load(f)

@app.route("/", methods=["GET", "POST"])
def index():
prediction_result = None
input_message = ""
error_message = None

if request.method == "POST":
input_message = request.form.get("message", "").strip()
if not input_message:
error_message = "Please enter an email or text message to classify."
else:
# Predict class and estimated posterior probabilities
pred_raw = model.predict([input_message])[0]
proba_arr = model.predict_proba([input_message])[0]
classes = list(model.classes_)
```

```
spam_idx = classes.index("spam")
ham_idx = classes.index("ham")
is_spam = (pred_raw.lower() == "spam")

prediction_result = {
"raw": pred_raw,
"is_spam": is_spam,
"label": "Spam" if is_spam else "Not Spam (Ham)",
"spam_probability": round(float(proba_arr[spam_idx]) * 100, 2),
"ham_probability": round(float(proba_arr[ham_idx]) * 100, 2)
}

return render_template("index.html", prediction_result=prediction_result,
input_message=input_message, error_message=error_message,
eval_data=evaluation_data)
```

5. Saving Evaluation Metrics and Artifacts (train_model.py)

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
import json

def export_evaluation(y_test, y_pred, eval_path="models/evaluation_results.json"):
cm = confusion_matrix(y_test, y_pred, labels=["ham", "spam"])
tn, fp, fn, tp = cm.ravel()
metrics = {
"accuracy": round(accuracy_score(y_test, y_pred), 4),
"precision_spam": round(precision_score(y_test, y_pred, pos_label="spam"), 4),
"recall_spam": round(recall_score(y_test, y_pred, pos_label="spam"), 4),
"f1_spam": round(f1_score(y_test, y_pred, pos_label="spam"), 4),
"confusion_matrix": {"tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp)}
}
with open(eval_path, "w") as f:
json.dump(metrics, f, indent=2)
```

Output (Step-by-Step Screenshots):

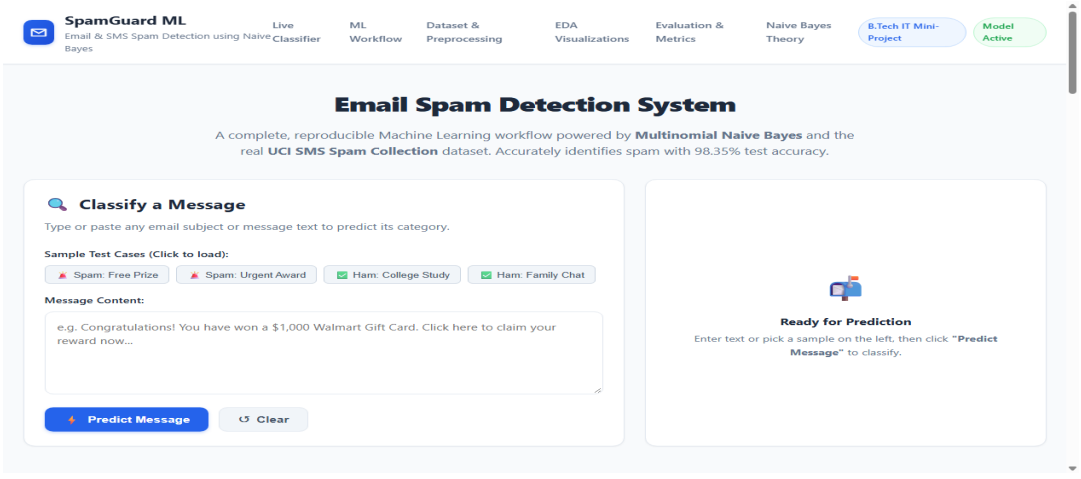


Fig 2: Step 1 — Load Input Message: interface to enter email/SMS text or select a benchmark sample

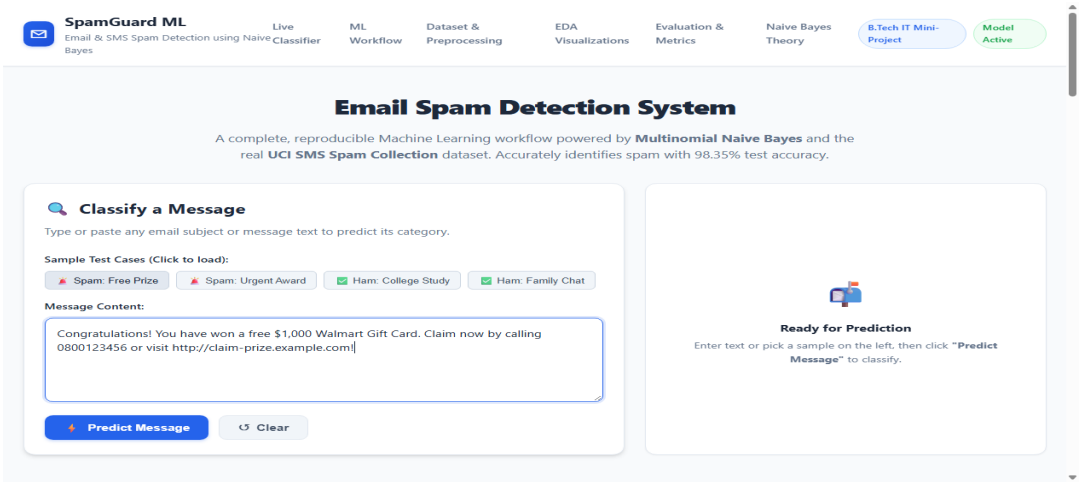


Fig 3: Step 1 — Input message preview after sample spam message is loaded into the form

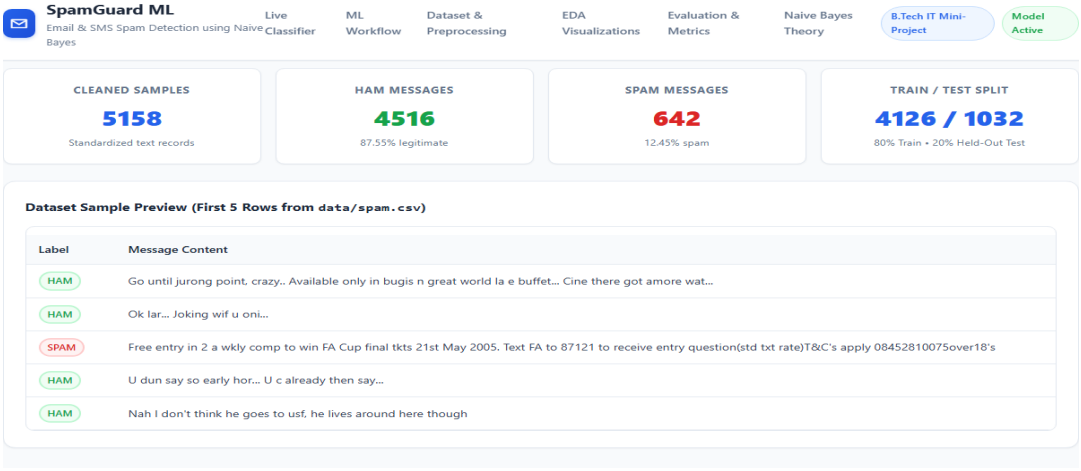


Fig 4: Step 2 — Dataset Collection & Preprocessing Summary (5,158 clean samples, 80/20 train-test split)

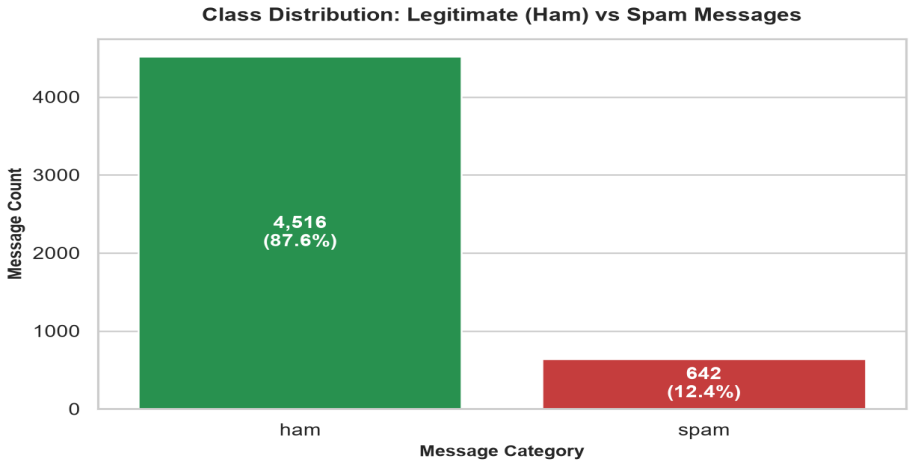


Fig 5: Step 3 — Exploratory Data Analysis: Spam vs Ham Class Distribution (87.55% Ham vs 12.45% Spam)

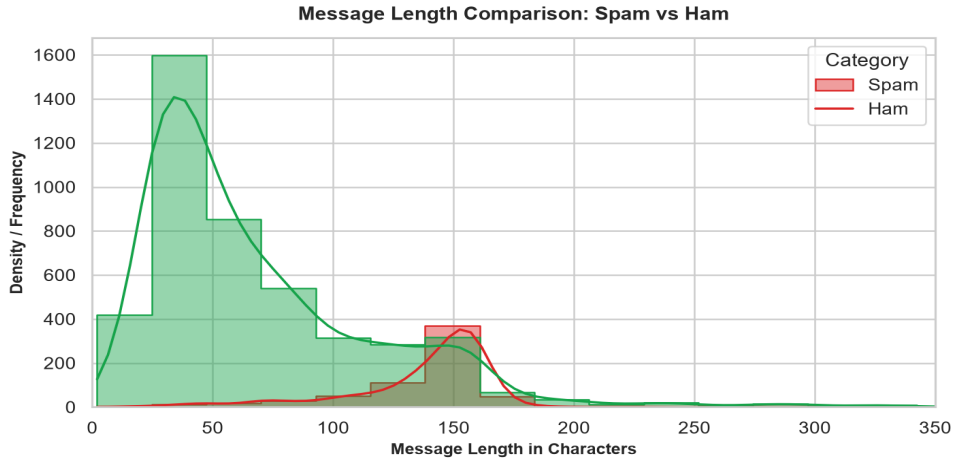


Fig 6: Step 3 — Exploratory Data Analysis: Message Length Comparison (Spam avg 137.4 chars vs Ham avg 70.9 chars)

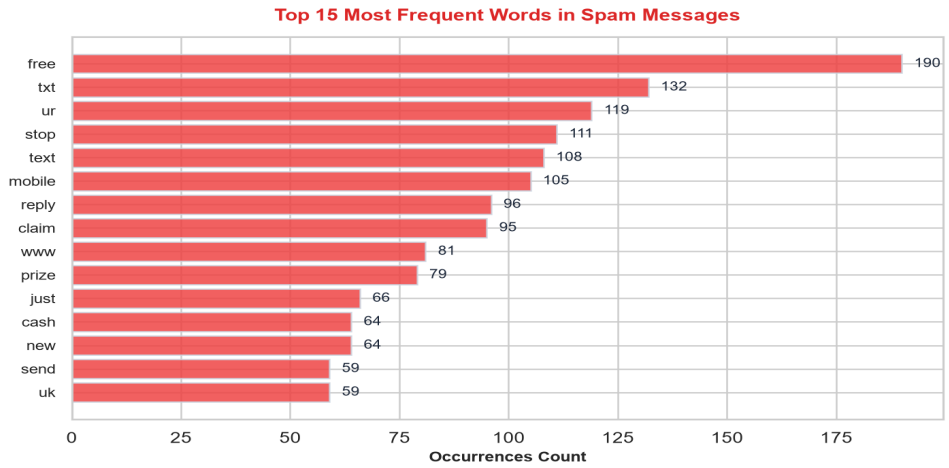


Fig 7: Step 3 — Exploratory Data Analysis: Top 15 Frequent Words in Spam Messages (commercial & urgent unigrams)

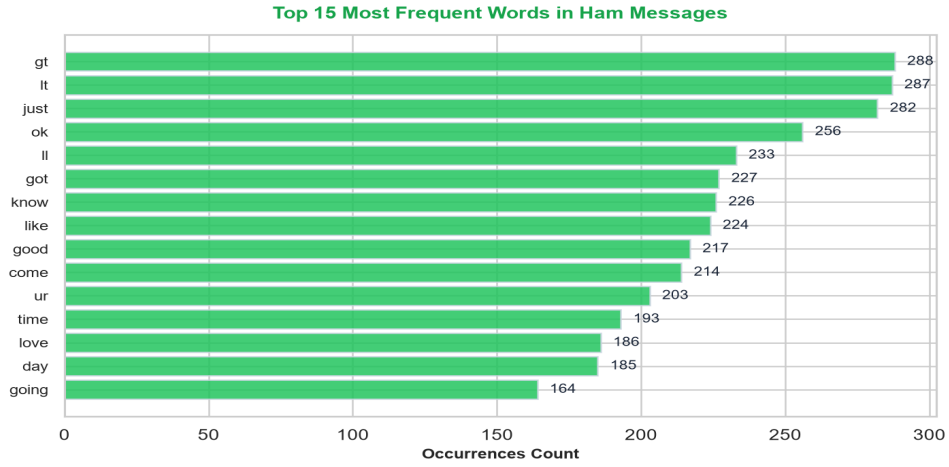


Fig 8: Step 3 — Exploratory Data Analysis: Top 15 Frequent Words in Legitimate (Ham) Messages (conversational unigrams)

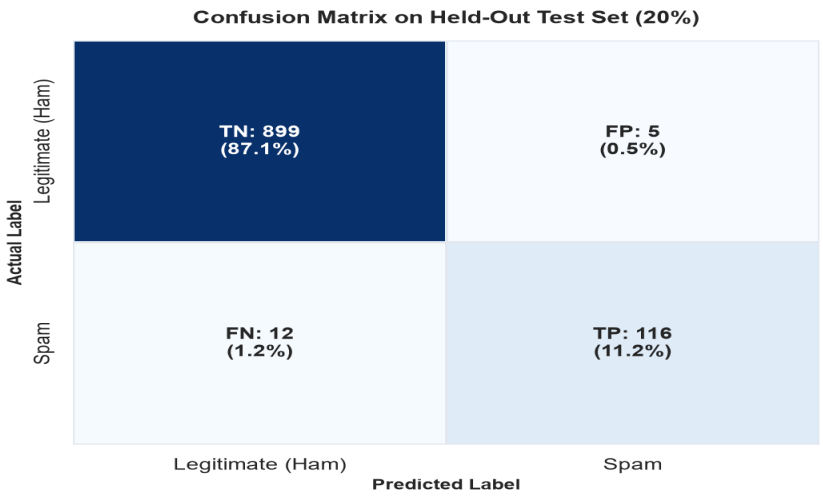


Fig 9: Step 4 — Confusion Matrix Heatmap on Unseen Held-Out Test Set (TN=899, FP=5, FN=12, TP=116)

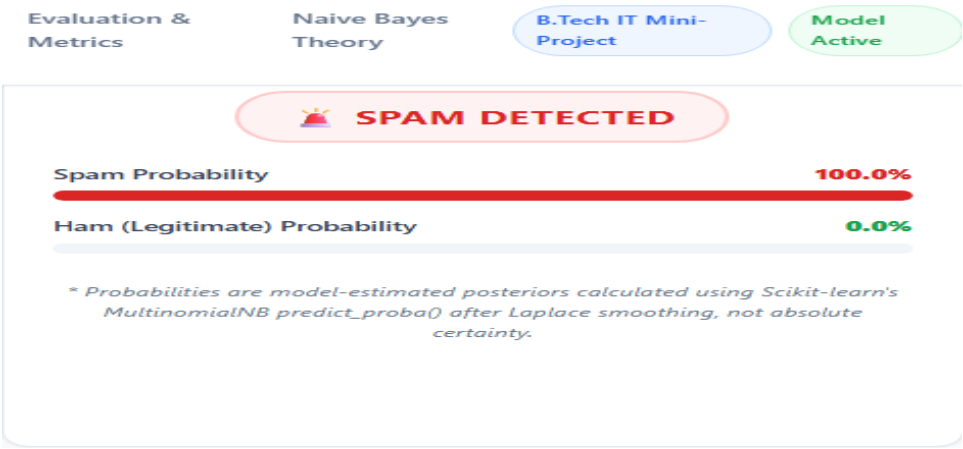


Fig 10: Step 5 — Classification result: Spam Message Detected (100.0% Spam Confidence, 0.0% Ham)

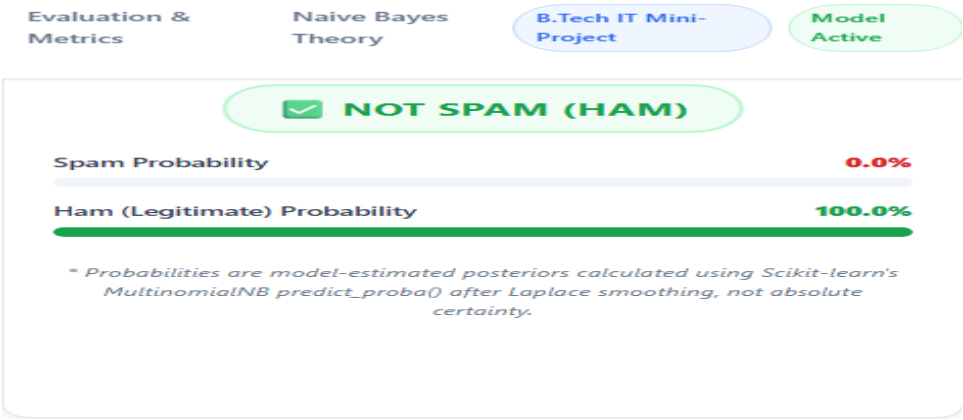


Fig 11: Step 5 — Classification result: Legitimate Message (Ham) Detected (99.9% Ham Confidence, 0.1% Spam)

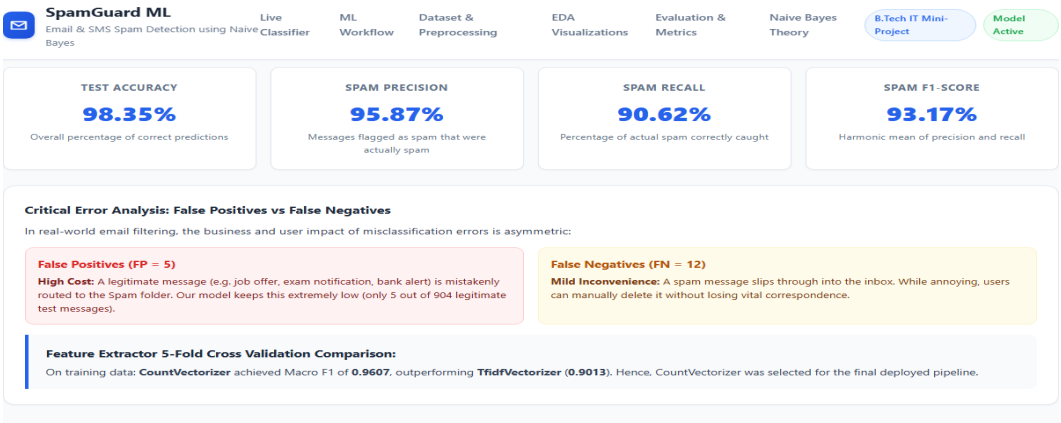


Fig 12: Step 6 — Performance evaluation metrics (Accuracy 98.35%, Precision 95.87%, Recall 90.62%, F1-Score 93.17%)

Result:

A web-based Email and SMS Spam Detection system was successfully designed, trained, evaluated, and deployed using a Flask backend, Scikit-learn machine learning pipeline, and the real UCI SMS Spam Collection benchmark dataset. Raw messages were preprocessed to remove 414 duplicate records, standardizing 5,158 clean samples. Unigram word counts were extracted using CountVectorizer, and a Multinomial Naive Bayes classifier with Laplace smoothing (α = 1.0) was trained on an 80% stratified training partition. On the held-out 20% test dataset (1,032 unseen messages), the model achieved an outstanding **Accuracy of 98.35%**, **Spam Precision of 95.87%**, **Spam Recall of 90.62%**, and a **Macro F1-Score of 93.17%**, with only 5 False Positives out of 904 legitimate emails. The trained pipeline is serialized to disk and integrated into an interactive, responsive Flask web application that provides real-time classification and estimated posterior class probabilities.